This attack focuses on fixing a user's session id known to the attacker, and forcing the user's browser into using this id. It is therefore not necessary for the attacker to steal the session id afterwards. Here is how this attack works:

- The attacker creates a valid session id: They load the login page of the web application where they want to fix the session, and take the session id in the cookie from the response (see number 1 and 2 in the image).
- They maintain the session by accessing the web application periodically in order to keep an expiring session alive.
- The attacker forces the user's browser into using this session id (see number 3 in the image). As you may not change a cookie of another domain (because of the same origin policy), the attacker has to run a JavaScript from the domain of the target web application. Injecting the JavaScript code into the application by XSS accomplishes this attack. Here is an example: `<script>document.cookie="_session_id=16d5b78abb28e3d6206b60f22a03c8d9";</script>`. Read more about XSS and injection later on.
- The attacker lures the victim to the infected page with the JavaScript code. By viewing the page, the victim's browser will change the session id to the trap session id.
- As the new trap session is unused, the web application will require the user to authenticate.
- From now on, the victim and the attacker will co-use the web application with the same session: The session became valid and the victim didn't notice the attack.

2.8 Session Fixation - Countermeasures

One line of code will protect you from session fixation.

The most effective countermeasure is to *issue a new session identifier* and declare the old one invalid after a successful login. That way, an attacker cannot use the fixed session identifier. This is a good countermeasure against session hijacking, as well. Here is how to create a new session in Rails:

```
reset_session
```

If you use the popular RestfulAuthentication plugin for user management, add reset_session to the SessionsController#create action. Note that this removes any value from the session, *you have to transfer them to the new session*.

Another countermeasure is to *save user-specific properties in the session*, verify them every time a request comes in, and deny access, if the information does not match. Such properties could be the remote IP address or the user agent (the web browser name), though the latter is less user-specific. When saving the IP address, you have to bear in mind that there are Internet service providers or large organizations that put their users behind proxies. *These might change over the course of a session*, so these users will not be able to use your application, or only in a limited way.

2.9 Session Expiry

Sessions that never expire extend the time-frame for attacks such as cross-site request forgery (CSRF), session hijacking and session fixation.

One possibility is to set the expiry time-stamp of the cookie with the session id. However the client can edit cookies that are stored in the web browser so expiring sessions on the server is safer. Here is an example of how to *expire sessions in a database table*. Call `Session.sweep("20 minutes")` to expire sessions that were used longer than 20 minutes ago.

```
class Session < ActiveRecord::Base
  def self.sweep(time = 1.hour)
    if time.is_a?(String)
```